Finding the Decisive Token Is Not Enough: Rank-One Self-Edits Repair Code Locally but Do Not Transfer

Avijit Bhuin
Independent Researcher

Abstract

Can a language model repair its own failures by directly editing its weights? We study this question on a small code model (Qwen3.5-0.8B) and MBPP+, using edits that need no gradients. In a pilot, the model chose neurons to rescale after a failed attempt. It fixed 5 of 45 failures, exactly as many as random rescaling and fewer than plain resampling (8). We then replace model-chosen edits with a precise diagnosis. Given a failing greedy answer and a passing sample, a binary search over forced prefixes finds the *decisive token*: the single next-token decision whose forcing flips the outcome from fail to pass. A rank-one MLP edit then flips that decision. Decisive-token edits fix 50 of 91 failing tasks, but edits at *random* positions where greedy disagrees with the passing answer fix 47 (McNemar p=0.72). Restricting edits to code tokens exposes a real but local signal: one decisive edit fixes 36 tasks versus 22 for one random-position edit (p=0.004). This signal does not transfer. Summing the fixing edits from a random half of the tasks and evaluating on the other half gives net held-out changes indistinguishable from random-position edits (+3.7 vs. +3.0 tasks at quarter scale, averaged over three splits), with 10–25 held-out tasks flipping in each direction. At full scale both destroy most passing tasks, while norm-matched random noise is harmless. We conclude that, in this setting, per-task weight edits repair a task mostly by re-routing its trajectory rather than by correcting a reusable mechanism, and that localization alone does not yield transferable self-improvement.

1 Introduction

Language models can increasingly inspect and modify neural networks, which raises an old question in a new form: can a model improve itself by editing its own weights? Knowledge editing shows that individual facts can be rewritten with small, targeted updates [Meng et al., 2022, 2023, Mitchell et al., 2022, Fang et al., 2025]. Self-adapting models show that a model can generate its own fine-tuning data and learn to do so with reinforcement learning [Zweiger et al., 2025]. Both lines suggest a tempting recipe for *skills* rather than facts: when the model fails a task, locate the faulty computation and edit it directly.

We test this recipe in its simplest form, on code generation, where success is checked automatically by hidden tests. Our question is not whether an edit can make a single task pass. With enough freedom almost any perturbation can. The question is whether locating the right place to edit buys anything over editing somewhere else, and whether the resulting edits generalize to tasks they were not built on.

Contributions.

- A deterministic *decisive-token* search: a binary search over forced prefixes of a passing sample that finds the single decision separating a failing greedy answer from a passing one. It requires no sampling and about $\log_2 L$ greedy rollouts.

- A controlled comparison of rank-one edits at decisive tokens against three controls: random positions, random directions, and norm-matched noise. We evaluate both per-task repair and transfer to held-out tasks over three random splits.
- A negative result with one positive nuance. Decisive tokens carry a real per-task signal (single-edit fixes, p=0.004), but it does not transfer, and accumulated edits along token-output directions are destructive even at modest norms.

2 Setup

Model and tasks. We use Qwen3.5-0.8B [Qwen Team, 2026] in bf16 on Apple MLX, with thinking mode disabled and greedy decoding (at most 1024 new tokens). Tasks are the 378 problems of MBPP+ [Austin et al., 2021, Liu et al., 2023]. The prompt shows the problem and its three original asserts (*visible tests*) and asks for the function only, without comments, docstrings, or tests. An answer *passes* if it passes both the visible tests and the full MBPP+ hidden suite. We exclude task 590, whose reference solution fails its own hidden suite, leaving 377 tasks. Greedy decoding passes 130 of them.

Pairs. For each task we draw 10 samples at temperature 0.7. A *pair* is a task whose greedy answer fails while at least one sample passes; there are 91. Among pairs, the mean sample pass rate is 0.27 (median 0.2). For each pair we choose, as the reference passing answer s, the passing sample that shares the longest prefix with the greedy answer.

Edit primitive. Let $a \in \mathbb{R}^{d_{\text{ff}}}$ be the input to the MLP down-projection W of layer ℓ at the last position of a context, and let r and w be the desired and the currently preferred next tokens. With unembedding E and final-norm gain g, we set $u = \frac{(E_r - E_w) \odot g}{\|(E_r - E_w) \odot g\|}$ and apply

$$W \leftarrow W + \alpha \, \frac{u \, a^\top}{\|a\|^2}, \tag{1}$$

which adds exactly αu to the MLP output for input a. Starting from α=1, we double α until r beats w by at least one logit at that context. We use ℓ=16 throughout. This is a deliberately simple, ROME-like rank-one update [Meng et al., 2022]. It has no key covariance, so it sets a lower bound on how local such an edit can be.

3 Pilot: the model chooses its own edit

Our first design gave the model control. After a failed attempt on the visible tests, we recorded the most active MLP neurons in layers 16–23 on the failing reply and labelled each with the tokens it promotes and suppresses under the logit lens [nostalgebraist, 2020, Geva et al., 2021]. We then asked the model to propose up to five neuron rescalings as JSON. Each edit was applied and the answer regenerated. Up to three rounds were allowed, and the weights were restored after every task. The control applied random rescalings to the same candidate neurons on the same schedule. Table 1 shows the outcome on the first 100 tasks. Model-chosen edits match random edits exactly and lose to simple resampling. The model cannot read its own neuron labels well enough to pick useful edits. The rest of the paper therefore asks a more basic question: does *any* localization, even a perfect external one, help?

Method (3 attempts)	Fixed / 45 failures
Resample (T=0.7)	8
Error feedback in context	3
Model-chosen neuron scaling	5
Random neuron scaling	5

Table 1: Pilot on the first 100 MBPP+ tasks. The denominator is greedy answers that fail the visible tests; a fix means the final answer passes the hidden suite.

4 Locating the decisive token

Let y be the failing greedy answer and s the reference passing answer. For $k \in \{0, \ldots, |s|\}$, let $G(k)$ be the outcome of forcing the prefix $s_{<k}$ and continuing greedily. By construction $G(0)$ fails and $G(|s|)$ passes. Binary search finds an index k^* with $G(k^*)$ failing and $G(k^*{+}1)$ passing, using about $\log_2 |s|$ greedy rollouts. At k^* the model's greedy choice w differs from $s_{k^*} = r$, and taking r instead is enough to turn the outcome. This is a deterministic, single-trajectory relative of pivotal-token search [Abdin et al., 2024], which estimates success probabilities by sampling.

The simpler alternative, the first position where y and s diverge, almost always lands on cosmetic tokens such as comment wording or docstring phrasing. We found the same for sampling-based pivotal tokens, which motivated both the code-only prompt and the decisive search.

Episodes. An episode applies up to three edits. At each step we find the decisive token under the *current* weights, flip it with the primitive above, keep the edit, and test whether greedy decoding from the bare prompt now passes. The episode stops early if the decisive search finds no bracket, or if r and w decode to the same text (e.g. two tokenizations of `int`). All edits are rolled back after the episode.

Controls. *Random position*: one forward pass on the prompt plus s lists every position where the model's argmax disagrees with s. We pick one uniformly at random and flip it in the same way. This control keeps the target answer, the direction family, and the stopping rule, and removes only the localization. *Random direction*: replacing u by a random unit vector never flipped a token within the α budget (0/8 in a smoke test), so we drop it as uninformative. *Noise* (held-out only): for every edit we replace u and a with random Gaussian vectors, with a rescaled to the original $\|a\|$ and the same α, so each rank-one term has the same norm. *Re-roll null*: a task with sample pass rate p that receives n edits would be fixed with probability $1 - (1 - p)^n$ if each edit amounted to drawing a fresh sample. Summed over tasks, this gives the fix count expected from rerolling alone.

Code-only variant. Even with the code-only prompt, about 35% of answers contain comments or docstrings. In the code-only variant we strip comments and docstrings from every passing sample, re-tokenize it, keep it only if it still passes, and then choose s among these. We also reject decisive tokens that fall inside a comment or docstring, outside the code fence, or that change only whitespace. Rejected tokens stop the episode and count as unfixed. The random-position control is restricted in the same way.

Edit location	Fixed / 91	Fixed by 1 edit	Re-roll null	McNemar p
all tokens				
Decisive token	50	31	35.2	0.72
Random position	47	22	42.2	–
code tokens only				
Decisive token	49	36	31.2	0.85
Random position	47	22	40.4	–

Table 2: Per-task repair on 91 pairs (greedy fails, ≥ 1 of 10 samples passes), with up to three rank-one edits per task at layer 16. The re-roll null is the expected number of fixes if each edit were a fresh sample. McNemar p compares the two rows of each block on paired task outcomes. In the single-edit column, the p-values are 0.078 (all tokens) and 0.004 (code tokens only).

5 Results

5.1 Per-task repair

Table 2 reports per-task repair on the 91 pairs. Decisive-token edits fix 50 tasks and random-position edits fix 47. The difference is not significant (exact McNemar on paired outcomes, p=0.72). Both beat the re-roll null (35 and 42 expected fixes), but only modestly. Random-position episodes use more edits (219 flips vs. 155), so their null is higher.

Restricting the comparison to the *first* edit is more informative. A single decisive edit fixes 31 tasks, against 22 for a single random-position edit (p=0.078). In the code-only variant the gap widens to 36 vs. 22 (p=0.004; still $p < 0.02$ after Bonferroni correction over the four tests reported here). Localization is therefore not useless: the decisive token is more often the one edit that is enough. However, random-position episodes catch up by their second or third edit, so total fixes end up equal. Any trajectory-altering flip has a fair chance of reaching *some* passing completion.

5.2 Transfer to held-out tasks

If a decisive edit corrects a reusable piece of computation, then the fixing edits from some tasks should help on others. We split the 377 tasks in half at random (seeds 0, 1, 2). We sum the edits of every fixed episode whose task is in the training half, multiply by a scale λ, and decode all tasks greedily in batches of 16. We compare each held-out task's outcome to the same batched decoding without edits. Batched and unbatched greedy decoding already disagree on 14 of 377 tasks, so changes of a few tasks are noise.

Table 3 gives the results. At λ=1 the summed edits are destructive. Decisive edits fix 6 held-out tasks and break 47; random-position edits fix 9 and break 42. Norm-matched noise changes almost nothing (5 fixed, 2 broken). The damage therefore comes from the *directions* u (token-output directions), not from the size of the update. At λ=0.25 net effects are small and inconsistent across seeds. Decisive edits average +3.3 (all tokens) and +3.7 (code only), against +0.3 and +3.0 for random positions. Every one of these comes with 7–25 broken tasks offset by a similar number of fixed ones. At λ=0.5 every configuration is net negative. Even on the *training* tasks, the summed edits recover only about half of the individual fixes (e.g. 12 of 21). The edits interfere with one another.

Tokens	Edit	×1.0 fixed/broke	×0.25 net (seeds 0/1/2)	×0.5 net
all	Decisive	6/47	+6 / +7 / -3	–
all	Random pos.	9/42	-5 / +1 / +5	–
all	Noise	5/2	+0	–
code only	Decisive	–	-3 / +13 / +1	-10 / -13 / -21
code only	Random pos.	–	-7 / +8 / +8	-15 / -5 / -12
code only	Noise	–	+3 / -3 / -1	–

Table 3: Held-out transfer. The fixing edits of training-half tasks are summed, scaled, and applied, and the held-out half is decoded greedily. "Fixed/broke" counts held-out tasks that change from fail to pass or from pass to fail (seed 0). "Net" is fixed minus broke for seeds 0/1/2. A dash means the configuration was not run.

6 Discussion

Repair without understanding. Our results fit a simple picture. A rank-one edit that flips one decision moves the greedy trajectory onto a different path. For about half of the tasks where some sample already passes, some nearby path passes. Picking the decisive decision makes the first jump more likely to land on a passing path, which is a real signal. But it does not identify a component whose correction carries over to other tasks. This mirrors the finding of Hase et al. [2023] for factual knowledge, where the location found by causal tracing does not predict where editing works best. Here, the decision found by outcome-level localization does not predict transferable edits.

Implications for self-editing. The pilot shows that a small model cannot choose useful edits from logit-lens labels. The main experiment shows that even a perfect external choice of *where* to edit, given a known passing answer, gives local repairs and not skills. A self-improving system built on direct weight edits would need edit directions that reflect computation rather than output tokens, plus a way to consolidate many edits without interference. Methods that edit through training data, such as SEAL [Zweiger et al., 2025], or that constrain updates to protect preserved behavior, such as AlphaEdit [Fang et al., 2025], address parts of this. Our results suggest that localization alone does not.

7 Limitations

We study one small model (0.8B), one layer, one benchmark, and the simplest rank-one editor. Larger models, covariance-aware editors such as ROME and MEMIT, multi-layer edits, and other domains could all behave differently. The 91 pairs and three held-out splits give limited power, and batched decoding adds noise of about ± 7 tasks. Our edits require a passing sample, so they cannot help on tasks the model never solves (156 of 247 failures). We report every configuration we ran, including the ones that failed.

8 Conclusion

A binary search over forced prefixes finds the token decision that separates a failing answer from a passing one, and a rank-one edit can flip it. This repairs about half of the solvable failures of a 0.8B code model. Yet edits at random disagreement positions repair almost as many, and neither

kind of edit transfers to held-out tasks. In this setting, locating the decisive token is not enough to turn self-editing into self-improvement.

Reproducibility and AI assistance

All code, run logs, and edit deltas are available at
`https://github.com/avijitbhuin21/decisive-token-self-edits`.
Every number in this paper is regenerated from the logs by `scripts/paper_stats.py`. The experiments were run on a single Apple M5 laptop. An AI coding assistant helped write the experiment code and draft this paper. The author reviewed all analyses and takes responsibility for the content.

References

Marah Abdin et al. Phi-4 technical report, 2024.

Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, Henryk Michalewski, David Dohan, Ellen Jiang, Carrie Cai, Michael Terry, Quoc Le, and Charles Sutton. Program synthesis with large language models, 2021.

Junfeng Fang et al. AlphaEdit: Null-space constrained knowledge editing for language models. In *International Conference on Learning Representations*, 2025. arXiv:2410.02355.

Mor Geva, Roei Schuster, Jonathan Berant, and Omer Levy. Transformer feed-forward layers are key-value memories. In *Proceedings of EMNLP*, 2021.

Peter Hase, Mohit Bansal, Been Kim, and Asma Ghandeharioun. Does localization inform editing? Surprising differences in causality-based localization vs. knowledge editing in language models. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2301.04213.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? Rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2305.01210.

Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, 2022. arXiv:2202.05262.

Kevin Meng, Arnab Sen Sharma, Alex Andonian, Yonatan Belinkov, and David Bau. Mass-editing memory in a transformer. In *International Conference on Learning Representations*, 2023. arXiv:2210.07229.

Eric Mitchell, Charles Lin, Antoine Bosselut, Chelsea Finn, and Christopher D. Manning. Fast model editing at scale. In *International Conference on Learning Representations*, 2022. arXiv:2110.11309.

nostalgebraist. interpreting GPT: the logit lens. LessWrong blog post, 2020.

Qwen Team. Qwen3.5-0.8B. Model release; MLX conversion `mlx-community/Qwen3.5-0.8B-bf16`, 2026.

Adam Zweiger, Jyothish Pari, Han Guo, Ekin Akyürek, Yoon Kim, and Pulkit Agrawal. Self-adapting language models, 2025.